CSCE 548 – Project 4

Full System Test and Deployment Documentation

Student: Preston Sands
Course: CSCE 548
Project: Pokémon Card Tracker

Project Deployment and Full System Test

The Pokémon Card Tracker is a multi-tier application developed for CSCE 548. The system allows users to browse Pokémon card sets, view individual cards, and manage a personal inventory of collected cards.

The application follows an **n-tier architecture**, which separates the system into multiple layers that communicate with each other. The architecture used in this project includes the following layers:

- Client Layer (User Interface)
- Service Layer (FastAPI REST API)
- Business Logic Layer
- Data Layer (SQLite Database)

The **client layer** provides the user interface and allows users to interact with the application through a web browser.

The **service layer** exposes REST API endpoints that process client requests.

The **business logic layer** validates input and applies business rules before interacting with the database.

The **data layer** stores Pokémon card and inventory data using SQLite.

The purpose of this document is to provide detailed instructions explaining how to download, configure, build, run, and test the entire system. The document also demonstrates that all layers of the system operate correctly through full system testing.

Downloading the Project Repository

The source code for this project is stored in a GitHub repository.

To download the project, open a terminal and run the following commands:

```
git clone <repository link>
cd pokemon-card-tracker
```

This command downloads the repository and moves into the project directory.

(Screenshot: Terminal showing successful git clone and entry into project directory)

Alternatively, the repository can be downloaded from the GitHub website by selecting **Code** → **Download ZIP** and extracting the project files.

```
undyingtomb@LAPTOP-97NORLMP:~/CSCE-548$
```

Installing Required Software

Before running the project, ensure the following software is installed on the system:

- Python 3.x
- Git
- pip package manager
- SQLite
- Web browser (Chrome, Firefox, Edge)

The Python libraries required by this project include:

- FastAPI
- Uvicorn
- Requests
- Pydantic

These dependencies will be installed using the project's requirements file.

Creating and Activating the Virtual Environment

The project uses a Python **virtual environment** to isolate dependencies.

Navigate to the project directory and create the virtual environment:

python3 -m venv .venv

Activate the environment.

Linux / Mac:

source .venv/bin/activate

Windows:

.venv\Scripts\activate

When activated, the terminal prompt will display:

(.venv)

```
undyingtomb@LAPTOP-97NORLMP:~/CSCE-548$ source .venv/bin/activate
(.venv) undyingtomb@LAPTOP-97NORLMP:~/CSCE-548$
```

Installing Project Dependencies

Install the required Python packages by running:

pip install -r requirements.txt

If the requirements file is missing dependencies, install them manually:

pip install fastapi uvicorn requests

Database Setup

The application uses a **SQLite database** to store Pokémon card and inventory data.

Verify that the database file exists in the project directory:

ls *.db

The expected database file is:

pokemon_cards.db

```
(.venv) undyingtomb@LAPTOP-97NORLMP:~/CSCE-548$ ls
README.md  pokemon-card-tracker  pokemon_cards.db
```

Open the database using SQLite:

sqlite3 pokemon_cards.db

The SQLite prompt will appear:

sqlite>

To verify that the database contains the correct tables, run:

.tables

The database should contain tables similar to:

- sets
- cards
- conditions
- inventory_item

```
(.venv) undyingtomb@LAPTOP-97NORLMP:~/CSCE-548/pokemon-card-tracker$ sqlite3 pokemon_cards.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> .tables
card            card_condition  card_set        inventory       inventory_item
sqlite>
```

To verify that the database contains inventory data, run:

SELECT * FROM inventory_item;

This query should return existing inventory records.

```
(.venv) undyingtomb@LAPTOP-97NORLMP:~/CSCE-548/pokemon-card-tracker$ sqlite3 pokemon_cards.db
SQLite version 3.45.1 2024-01-30 16:01:20
Enter ".help" for usage hints.
sqlite> .tables
card            card_condition  card_set        inventory       inventory_item
sqlite> SELECT * FROM inventory_item;
2|2|1|0|1|PSA|9.0|1|55.0|2025-03-03|Zacian V graded
4|4|1|0|0|||1|2.0|2025-01-10|Rillaboom
5|5|2|0|0|||1|1.5|2025-01-10|Cinderace
6|6|2|0|0|||1|1.75|2025-01-10|Inteleon
7|7|1|0|0|||2|0.5|2025-01-10|Morpeko extras
8|8|1|0|0|||1|0.25|2025-01-10|Quick Ball
9|9|1|0|0|||1|0.25|2025-01-10|Rare Candy
10|10|2|0|0|||1|0.25|2025-01-10|Metal Saucer
11|11|2|0|0|||1|8.0|2025-04-10|Empoleon V
12|12|3|0|0|||1|6.0|2025-04-10|Tyranitar V
13|13|2|0|1|CGC|8.5|1|28.0|2025-05-01|Urshifu VMAX graded
14|14|2|0|0|||1|4.0|2025-04-10|Victini V
15|15|3|0|0|||1|7.0|2025-04-10|Victini VMAX
16|16|2|0|0|||1|16.0|2025-04-10|Rapid Strike Urshifu VMAX
17|17|1|0|0|||1|2.0|2025-04-11|Bruno full art
18|18|1|0|0|||1|3.0|2025-04-11|Cheryl full art
19|19|1|0|0|||2|0.25|2025-04-11|Level Ball extras
20|20|1|0|0|||1|0.25|2025-04-11|Tower of Waters
21|21|2|0|0|||1|20.0|2025-06-15|Galarian Moltres V
23|23|1|0|0|||1|3.0|2025-06-15|Melony full art
24|24|3|0|0|||1|10.0|2025-06-15|Galarian Articuno V
25|25|2|0|0|||1|14.0|2025-06-15|Shadow Rider Calyrex VMAX
26|26|2|0|0|||1|6.0|2025-06-15|Blaziken V
27|27|2|0|0|||1|13.0|2025-06-15|Blaziken VMAX
28|28|2|0|0|||1|9.0|2025-06-15|Galarian Zapdos V
29|29|1|0|0|||1|0.25|2025-06-16|Fog Crystal
30|30|2|0|0|||1|1.0|2025-06-16|Path to the Peak
31|31|2|0|0|||1|20.0|2025-06-20|Rayquaza V
32|32|2|0|0|||1|25.0|2025-06-20|Leafeon VMAX
33|33|2|0|1|BGS|9.5|1|180.0|2025-06-20|Umbreon VMAX graded
34|34|2|0|0|||1|12.0|2025-06-20|Sylveon V
35|35|3|0|0|||1|18.0|2025-06-20|Sylveon VMAX
36|36|3|0|0|||1|15.0|2025-06-20|Glaceon VMAX
37|37|1|0|0|||1|1.0|2025-06-21|Frosslass
```

Exit the SQLite interface by typing:

.quit

Running the Backend Service (API)

The service layer of the application is implemented using **FastAPI**.

Ensure the virtual environment is active, then start the API server by running:

uvicorn api:app --reload

If successful, the terminal will display a message similar to:

INFO: Uvicorn running on http://127.0.0.1:8000

```
INFO:     Will watch for changes in these directories: ['/home/undyingtomb/CSCE-548/pokemon-card-tracker']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [879] using StatReload
INFO:     Started server process [881]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

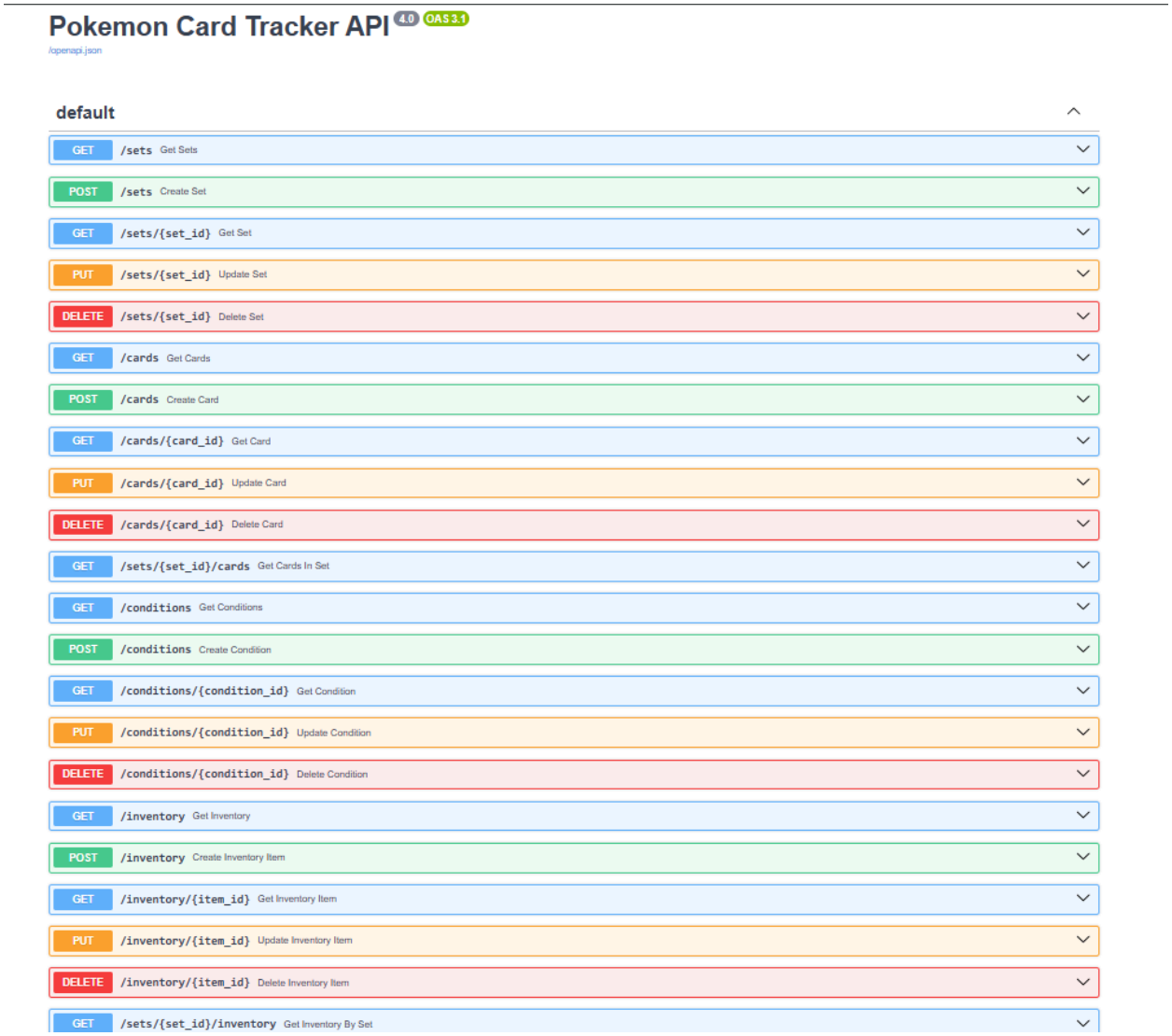
Accessing the API Documentation

FastAPI automatically generates interactive API documentation.

Open a browser and navigate to:

http://127.0.0.1:8000/docs

This page displays the Swagger interface for interacting with the API.



The Swagger interface allows the user to test API endpoints for sets, cards, conditions, and inventory records.

Running the Client Layer (Front End)

The client layer provides the graphical user interface used to interact with the Pokémon Card Tracker system. The client communicates with the backend API through HTTP requests.

Before starting the client, ensure that the backend API server is already running.

The backend server should be started using:

uvicorn api:app --reload

When the backend is running correctly, the terminal should display:

INFO: Uvicorn running on http://127.0.0.1:8000

Locating the Client Files

Navigate to the project folder containing the repository files.

Example:

cd pokemon-card-tracker

Cd project3-client

List the contents of the directory:

ls

You should see a client file named **index.html**.

```
undyingtomb@LAPTOP-97NORLMP:~/CSCE-548/pokemon-card-tracker/project3-client$ ls
index.html
```

The **index.html file is the client interface**.

Starting a Local Web Server for the Client

The client must be served from a local web server so that it can communicate with the API.

From the project directory, run:

python3 -m http.server 5500

This command starts a simple web server using Python on **port 5500**.

The terminal should display something similar to:

Serving HTTP on 0.0.0.0 port 5500 (http://0.0.0.0:5500/) ...

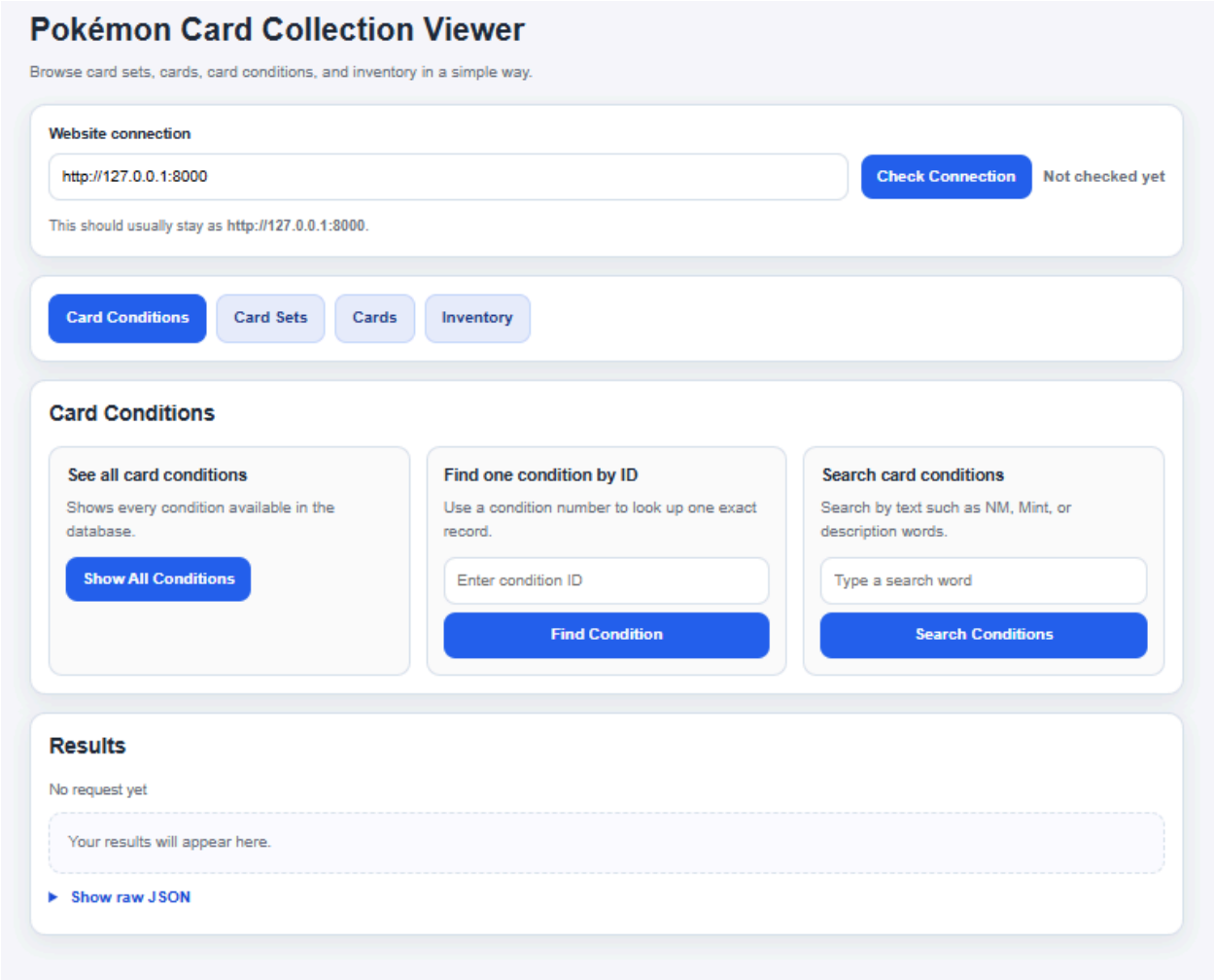
```
undyingtomb@LAPTOP-97NORLMP:~/CSCE-548/pokemon-card-tracker/project3-client$ python3 -m http.server 5500
Serving HTTP on 0.0.0.0 port 5500 (http://0.0.0.0:5500/) ...
```

Opening the Client Interface

Open a web browser and navigate to the following address:

http://127.0.0.1:5500

This loads the client interface from the local web server.



Verifying the Client is Connected to the API

The client communicates with the backend API running on port **8000**.

The API base URL is:

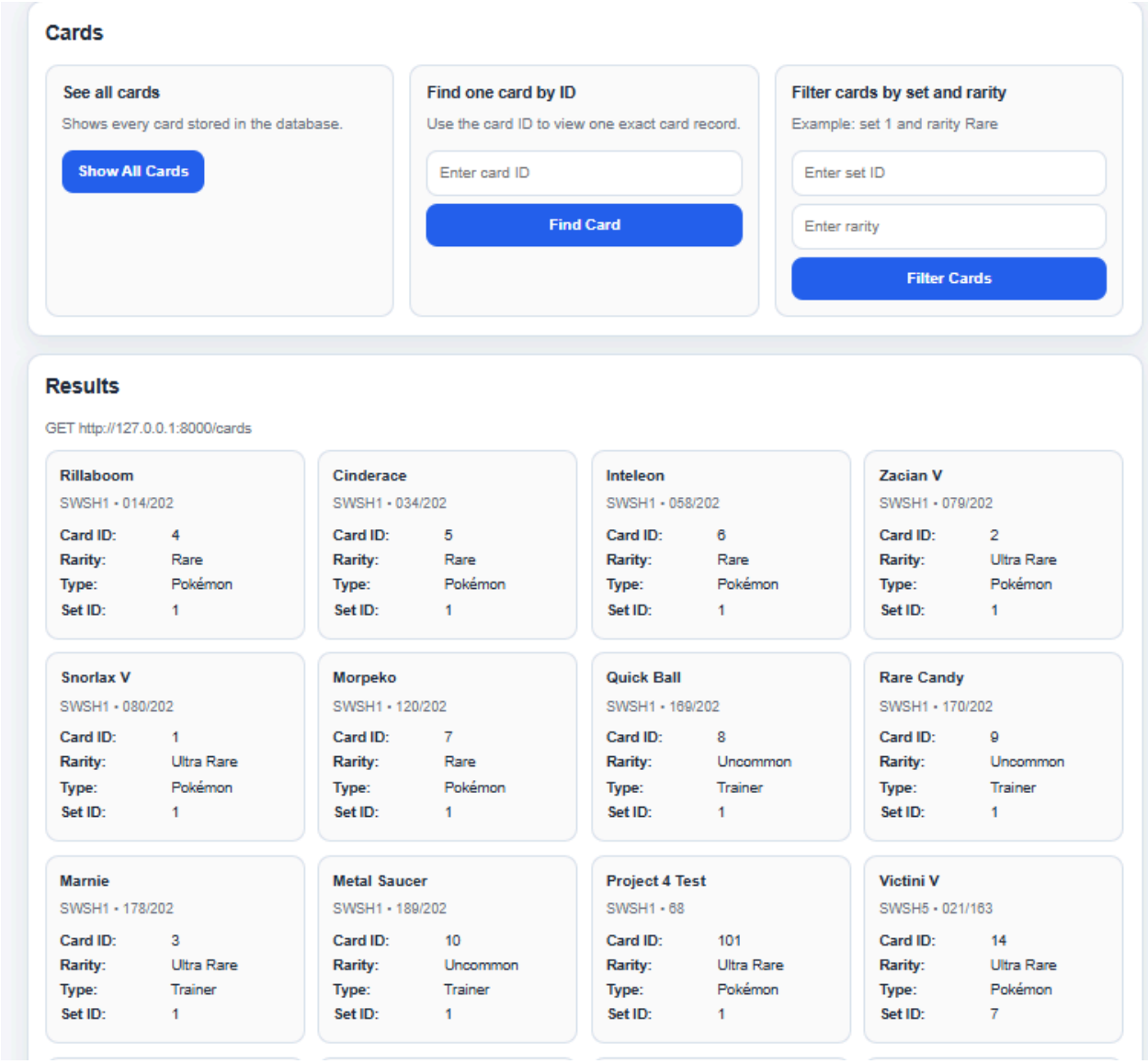
http://127.0.0.1:8000

When the client loads successfully, it automatically sends requests to the API to retrieve data.

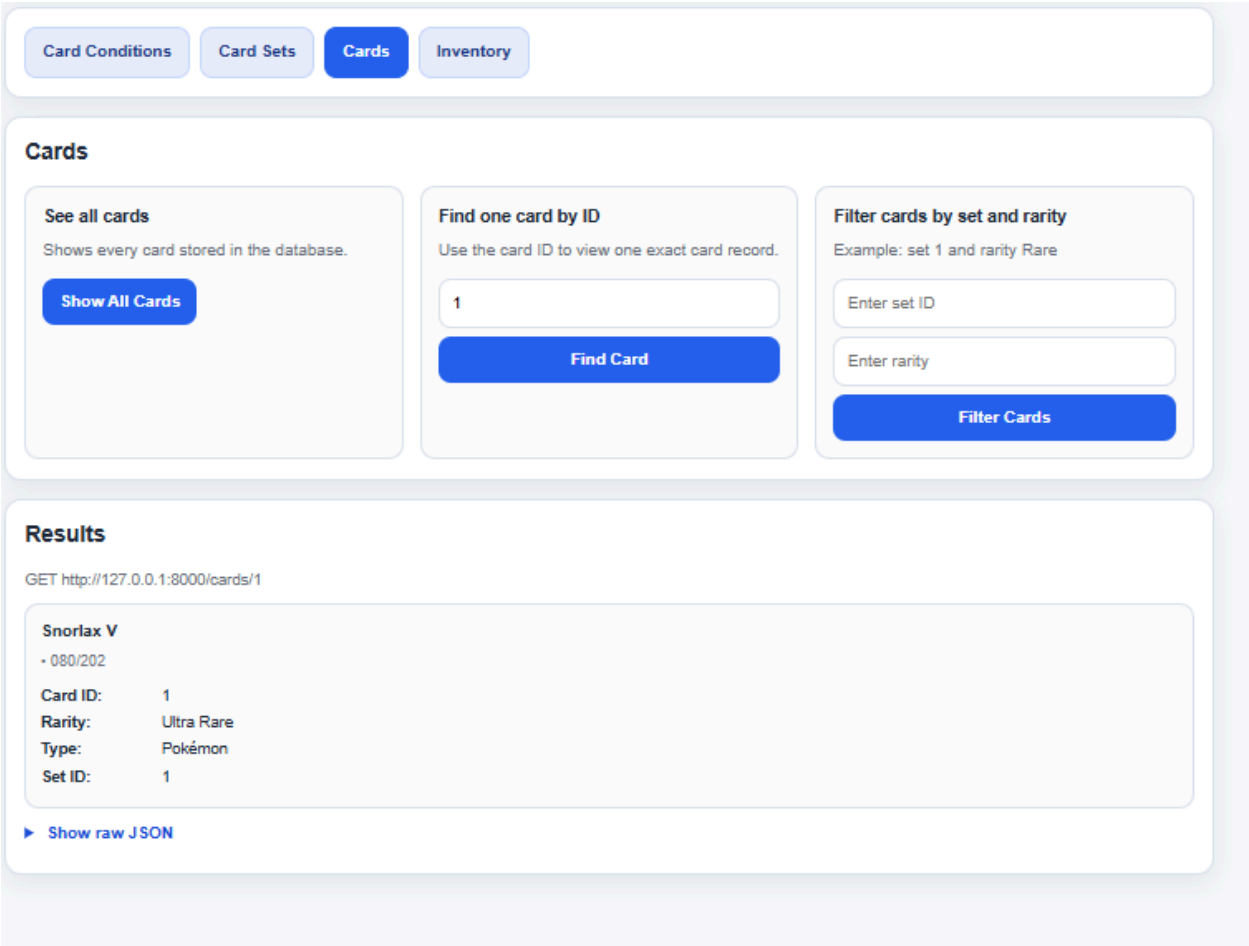
If everything is configured correctly, the following should occur:

- Pokémon card sets are displayed
- Individual cards load when a set is selected
- Inventory records appear in the interface

(Screenshot: Client displaying Pokémon card data retrieved from the API)



(Screenshot: Client displaying inventory records retrieved from the API)



Verifying Client Requests in the Browser Developer Tools

To confirm that the client is communicating with the API:

1. Open the browser developer tools
2. Select the **Network** tab
3. Refresh the page

You should see requests being sent to the API such as:

GET http://127.0.0.1:8000/cards

GET http://127.0.0.1:8000/inventory

These requests confirm that the client is retrieving data from the service layer.

(Screenshot: Browser developer tools showing API requests)

```
undyingtomb@LAPTOP-97NORLMP:~$ cd ~/CSCE-548/pokemon-card-tracker
source .venv/bin/activate
uvicorn api:app --reload
INFO:     Will watch for changes in these directories: ['/home/undyingtomb/CSCE-548/pokemon-card-tracker']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [1196] using StatReload
INFO:     Started server process [1198]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:42202 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:42202 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:35922 - "GET / HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:49956 - "GET /cards HTTP/1.1" 200 OK
INFO:     127.0.0.1:35106 - "GET /inventory HTTP/1.1" 200 OK
```

Confirming the Full System is Running

The entire system is functioning correctly when the following conditions are met:

1. The backend API server is running on port **8000**
2. The client web server is running on port **5500**
3. The client page loads successfully in the browser
4. Pokémon card and inventory data appear in the interface
5. API requests are visible in the browser developer tools

When these conditions are satisfied, it confirms that:

- The **client layer** is functioning
- The **service layer** is processing requests
- The **business logic layer** is handling validation
- The **data layer** is retrieving and storing information in the database

---

Full System Test

The following tests verify that the entire system works correctly across all layers. Each test includes verification from the client layer, service layer, and data layer.

Test 1 – Retrieve All Inventory Items

Purpose: Verify that the system can retrieve all inventory records.

Steps:

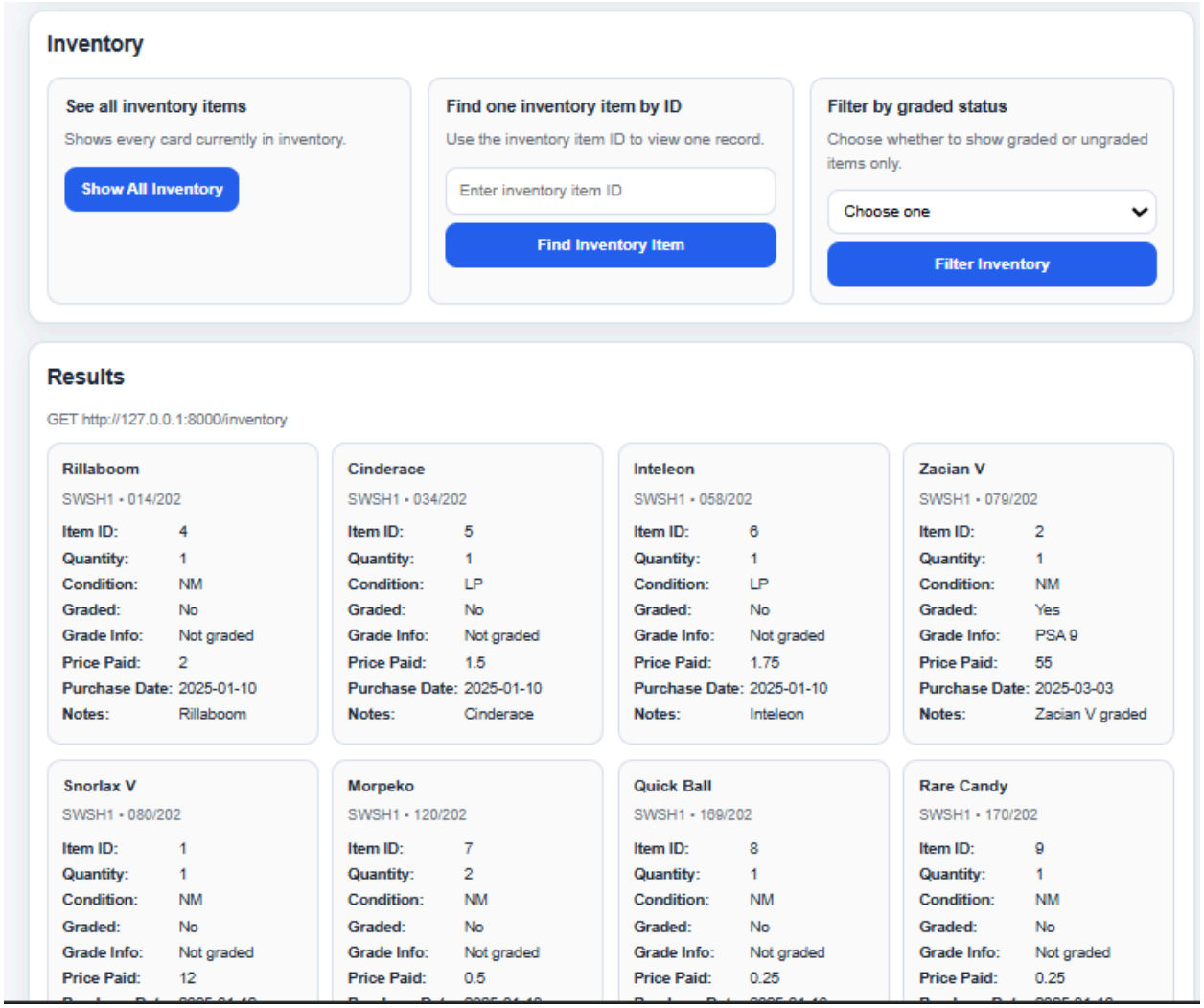
1. Open the client interface.
2. Navigate to the inventory section.
3. Request all inventory items.
4. Execute the GET /inventory API endpoint.
5. Query the inventory table in SQLite.

Expected Result:

The client displays all inventory items, the API returns matching JSON data, and the database contains the same records.

Verification:

(Screenshot: Client displaying full inventory list) (My inventory list to large to display in a screenshot but it is all there)



(Screenshot: Swagger interface showing GET /inventory request and response)

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/inventory' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/inventory
```

Server response

Code	Details
200	

Response body


```
[
  {
    "item_id": 4,
    "card_id": 4,
    "condition_id": 1,
    "is_foil": 0,
    "is_graded": 0,
    "graded_company": null,
    "grade": null,
    "quantity": 1,
    "purchase_price": 2,
    "purchase_date": "2025-01-10",
    "notes": "Rillaboom",
    "card_name": "Rillaboom",
    "card_number": "014/202",
    "rarity": "Rare",
    "set_code": "SWSH1",
    "set_name": "Sword & Shield Base",
    "condition_code": "NM"
  },
  {
    "item_id": 5,
    "card_id": 5,
    "condition_id": 2,
    "is_foil": 0,
    "is_graded": 0,
    "graded_company": null,
    "grade": null,
    "quantity": 1,
    "purchase_price": 1.5
```


Download

Response headers

```
content-length: 19996
content-type: application/json
date: Mon,16 Mar 2026 22:20:16 GMT
server: uvicorn
```

(Screenshot: SQLite query showing inventory_item table contents)

```
sqlite> SELECT * FROM inventory_item;
2|2|1|0|1|PSA|9.0|1|55.0|2025-03-03|Zacian V graded
4|4|1|0|0|||1|2.0|2025-01-10|Rillaboom
5|5|2|0|0|||1|1.5|2025-01-10|Cinderace
6|6|2|0|0|||1|1.75|2025-01-10|Inteleon
7|7|1|0|0|||2|0.5|2025-01-10|Morpeko extras
8|8|1|0|0|||1|0.25|2025-01-10|Quick Ball
9|9|1|0|0|||1|0.25|2025-01-10|Rare Candy
10|10|2|0|0|||1|0.25|2025-01-10|Metal Saucer
11|11|2|0|0|||1|8.0|2025-04-10|Empoleon V
12|12|3|0|0|||1|6.0|2025-04-10|Tyranitar V
13|13|2|0|1|CGC|8.5|1|28.0|2025-05-01|Urshifu VMAX graded
14|14|2|0|0|||1|4.0|2025-04-10|Victini V
15|15|3|0|0|||1|7.0|2025-04-10|Victini VMAX
16|16|2|0|0|||1|16.0|2025-04-10|Rapid Strike Urshifu VMAX
17|17|1|0|0|||1|2.0|2025-04-11|Bruno full art
18|18|1|0|0|||1|3.0|2025-04-11|Cheryl full art
19|19|1|0|0|||2|0.25|2025-04-11|Level Ball extras
20|20|1|0|0|||1|0.25|2025-04-11|Tower of Waters
21|21|2|0|0|||1|20.0|2025-06-15|Galarian Moltres V
23|23|1|0|0|||1|3.0|2025-06-15|Melony full art
24|24|3|0|0|||1|10.0|2025-06-15|Galarian Articuno V
25|25|2|0|0|||1|14.0|2025-06-15|Shadow Rider Calyrex VMAX
26|26|2|0|0|||1|6.0|2025-06-15|Blaziken V
27|27|2|0|0|||1|13.0|2025-06-15|Blaziken VMAX
28|28|2|0|0|||1|9.0|2025-06-15|Galarian Zapdos V
29|29|1|0|0|||1|0.25|2025-06-16|Fog Crystal
30|30|2|0|0|||1|1.0|2025-06-16|Path to the Peak
31|31|2|0|0|||1|20.0|2025-06-20|Rayquaza V
32|32|2|0|0|||1|25.0|2025-06-20|Leafeon VMAX
33|33|2|0|1|BGS|9.5|1|180.0|2025-06-20|Umbreon VMAX graded
34|34|2|0|0|||1|12.0|2025-06-20|Sylveon V
35|35|3|0|0|||1|18.0|2025-06-20|Sylveon VMAX
36|36|3|0|0|||1|15.0|2025-06-20|Glaceon VMAX
37|37|1|0|0|||1|1.0|2025-06-21|Frosslass
38|38|1|0|0|||1|0.25|2025-06-21|Stormy Mountains
39|39|2|0|0|||1|2.0|2025-06-21|Zinnia's Resolve
40|40|1|0|0|||1|0.25|2025-06-21|Switching Cups
41|41|1|0|0|||3|0.25|2025-07-04|Celebrations Pikachu extras
```

Test 2 – Retrieve Inventory Item by ID

Purpose: Verify that the system can retrieve a specific inventory record.

Steps:

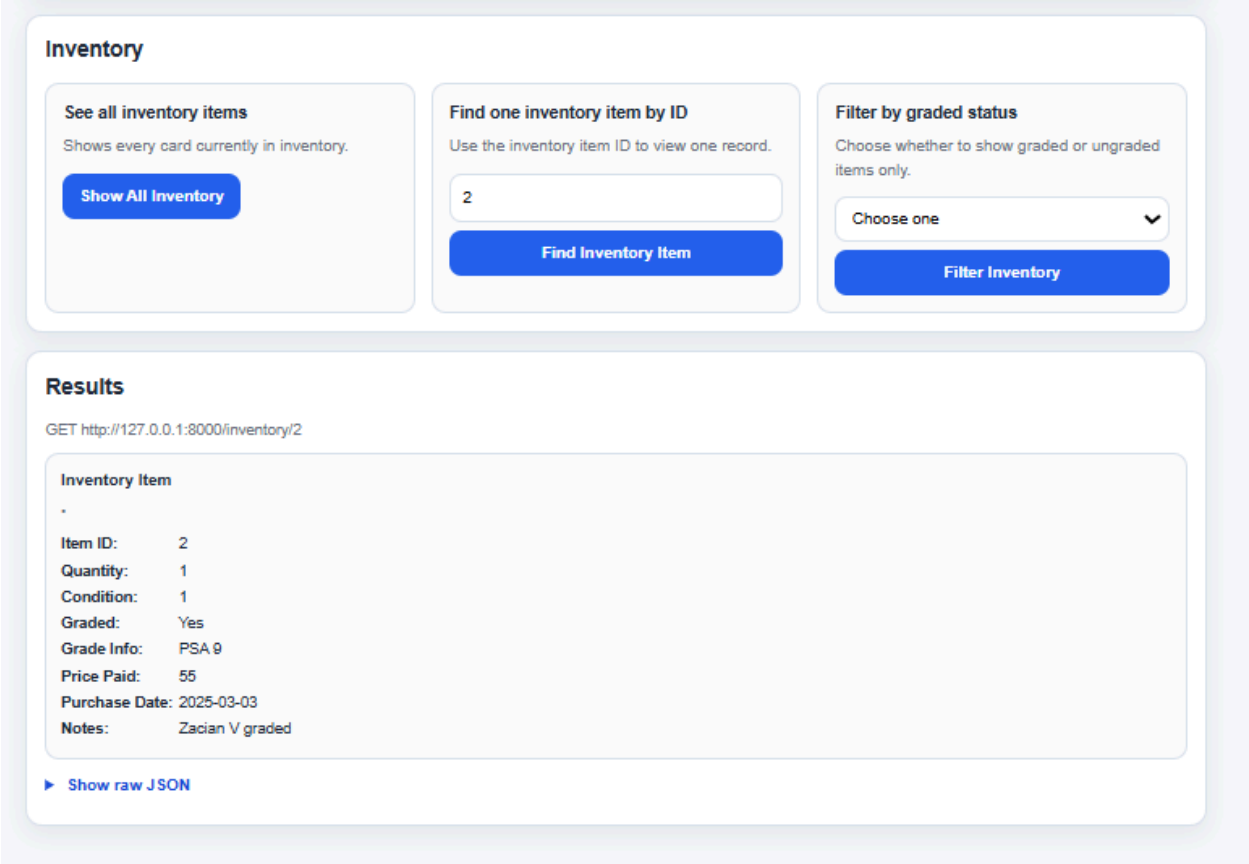
1. Select an inventory item from the client interface.
2. Use the item's ID to call the GET /inventory/{item_id} endpoint.
3. Query the database for the same item.

Expected Result:

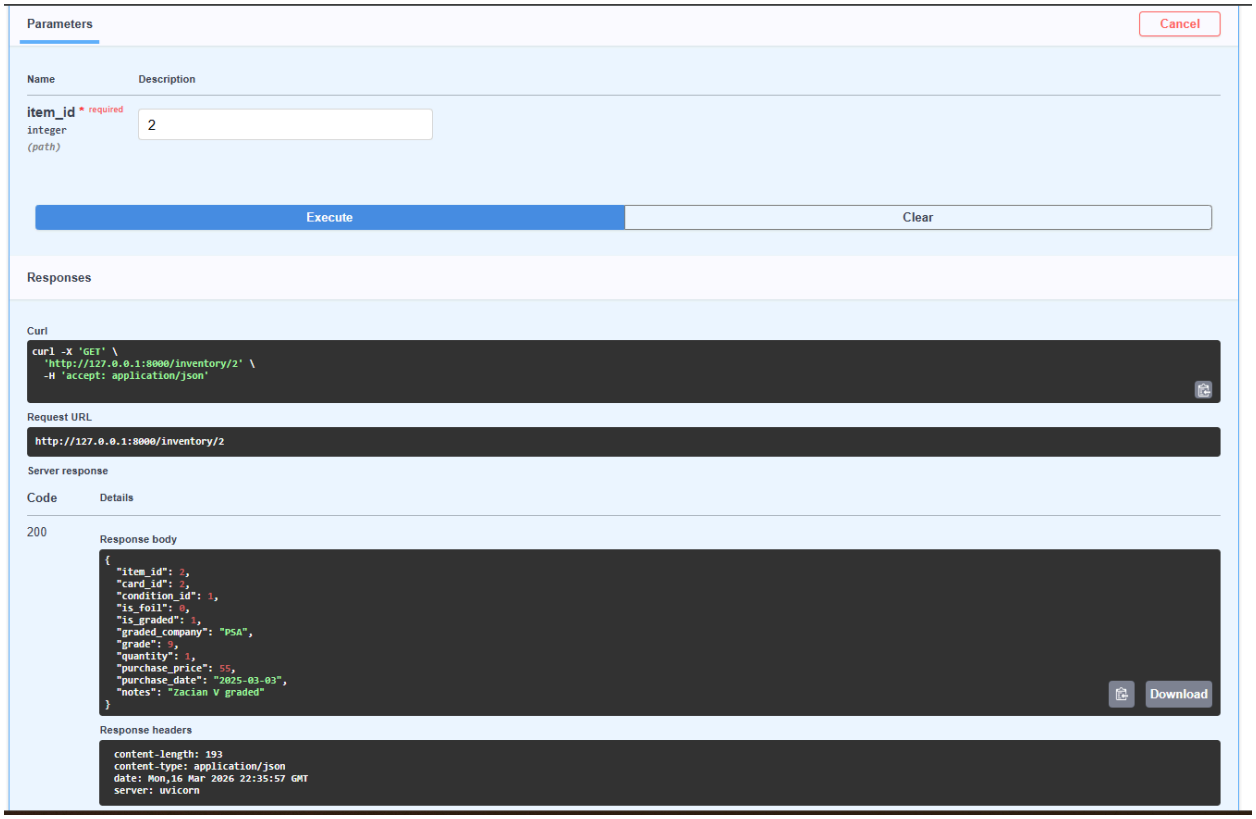
The client, API response, and database all display the same inventory record.

Verification:

(Screenshot: Client displaying selected inventory item)



(Screenshot: Swagger showing GET /inventory/{item_id} response)



(Screenshot: SQLite query showing matching inventory_item record)

```
sqlite> SELECT * FROM inventory_item WHERE item_id = 2;
2|2|1|0|1|PSA|9.0|1|55.0|2025-03-03|Zacian V graded
sqlite>
```

Test 3 – Insert a New Inventory Item

Purpose: Verify that the system can create new inventory records.

Steps:

1. Open the client interface.
2. Enter information for a new inventory item.

3. Submit the form.

4. Execute the POST /inventory API request.

5. Query the newest database record.

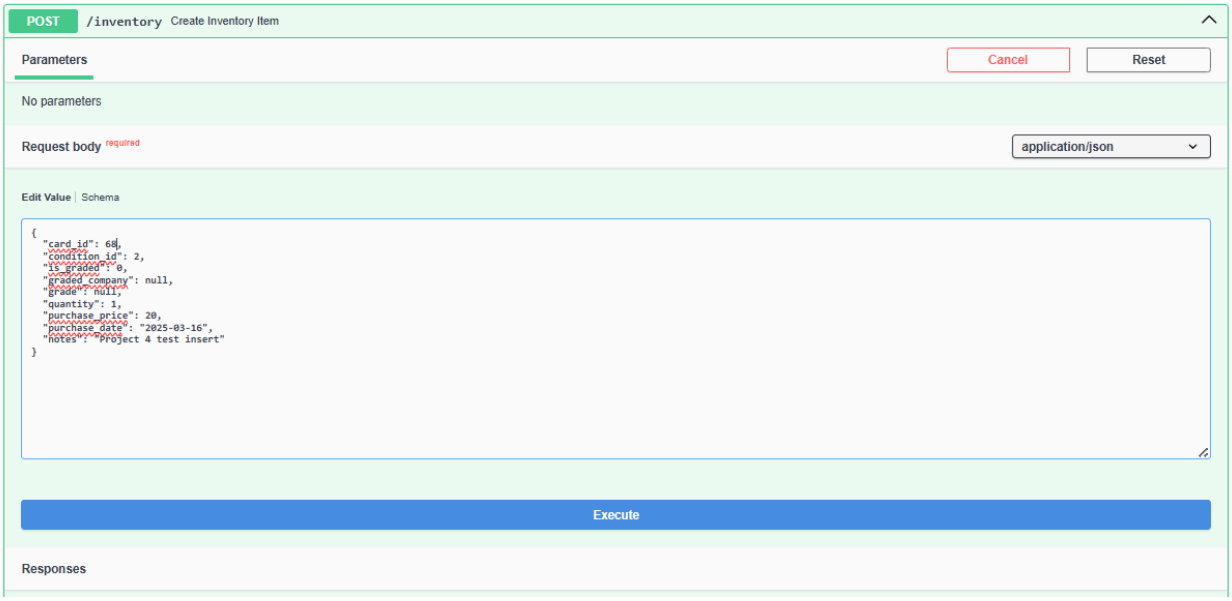
Expected Result:

The new item appears in the client interface, the API returns the created record, and the database contains the new row.

Verification:

(Screenshot: Swagger showing POST /card)

(Screenshot: Swagger showing POST /inventory request body before execution)



(Screenshot: Swagger showing successful POST /inventory response)

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/inventory' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "card_id": 68,
  "condition_id": 2,
  "is_graded": 0,
  "graded_company": null,
  "grade": null,
  "quantity": 1,
  "purchase_price": 20,
  "purchase_date": "2025-03-16",
  "notes": "Project 4 test insert"
}'
```

Request URL

```
http://127.0.0.1:8000/inventory
```

Server response

Code	Details
201	Response body

```
{
  "item_id": 61,
  "card_id": 68,
  "condition_id": 2,
  "is_foil": 0,
  "is_graded": 0,
  "graded_company": null,
  "grade": null,
  "quantity": 1,
  "purchase_price": 20,
  "purchase_date": "2025-03-16",
  "notes": "Project 4 test insert"
}
```

Download

Response headers

```
access-control-allow-credentials: true
content-length: 201
content-type: application/json
date: Mon,16 Mar 2026 22:50:20 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

422	Validation Error	*No links*

Media type

application/json

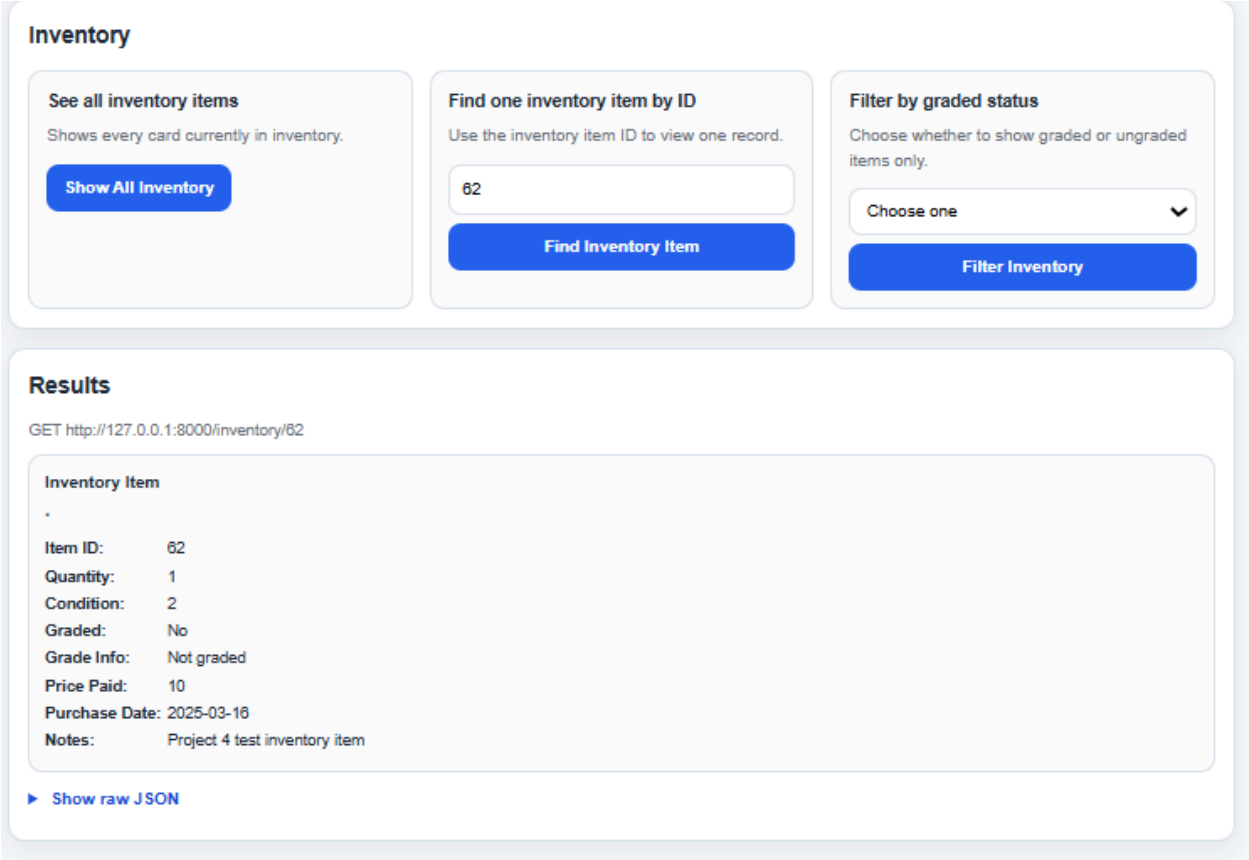
Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

(Screenshot: SQLite query showing newly inserted inventory_item row)

```
61|68|2|0|0|||1|20.0|2025-03-16|Project 4 test insert
```

(Screenshot: Client displaying newly created inventory item after insertion) (I had to restart so I placed a filler id 62 to replicate what I did for 61)



Test 4 – Update an Inventory Item

Purpose: Verify that an existing inventory record can be updated.

Steps:

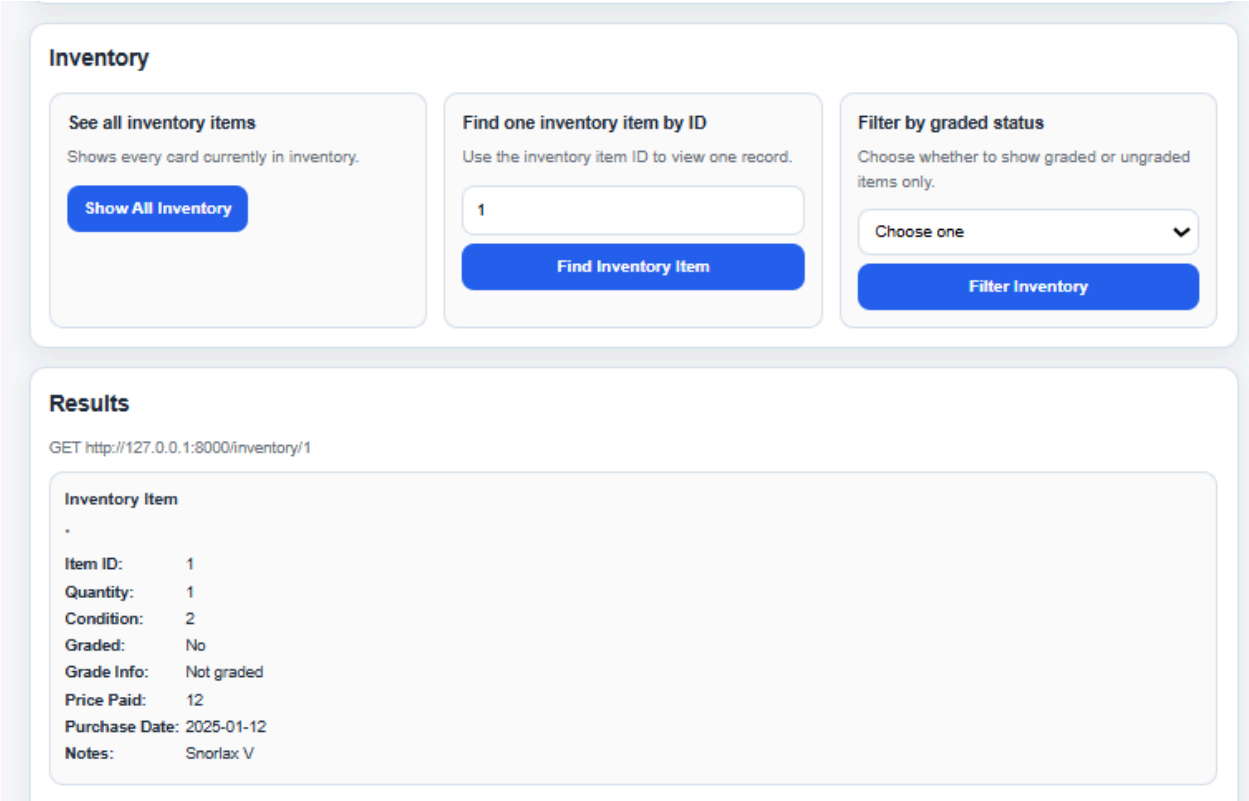
1. Select an inventory item in the client interface.
2. Modify one or more fields.
3. Submit the update.
4. Execute the PUT /inventory/{item_id} API request.
5. Query the same row in the database.

Expected Result:

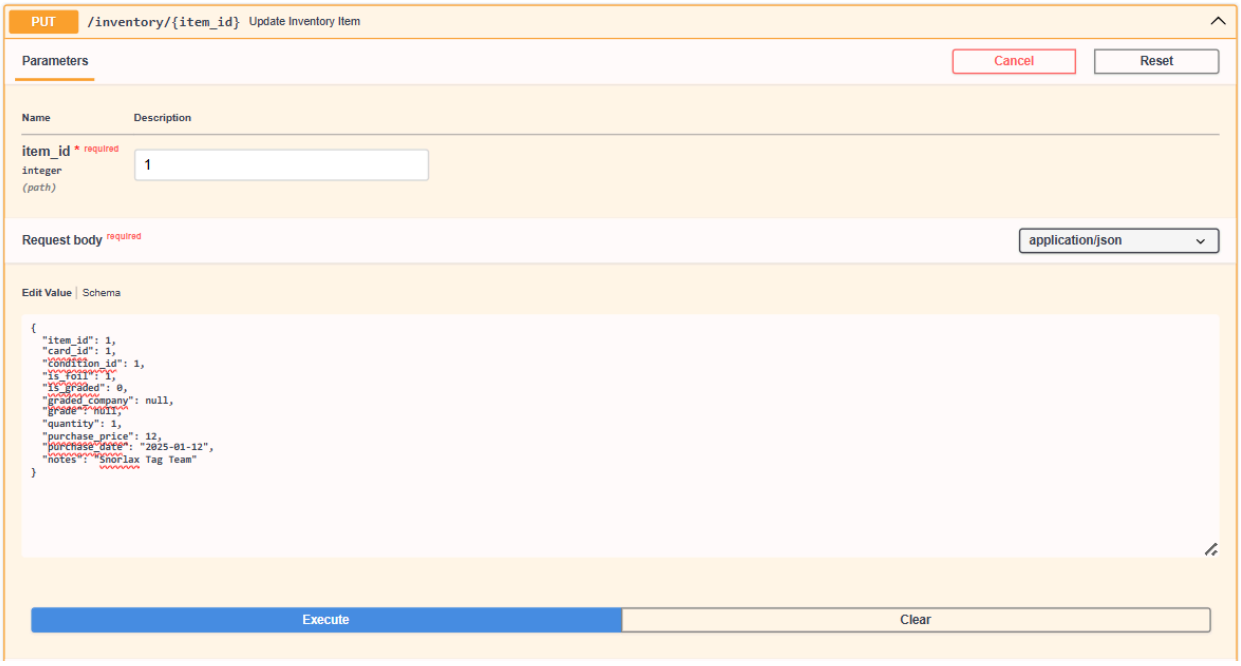
The updated values appear in the client interface, the API response reflects the update, and the database row contains the modified data.

Verification:

(Screenshot: Client displaying inventory item before update)



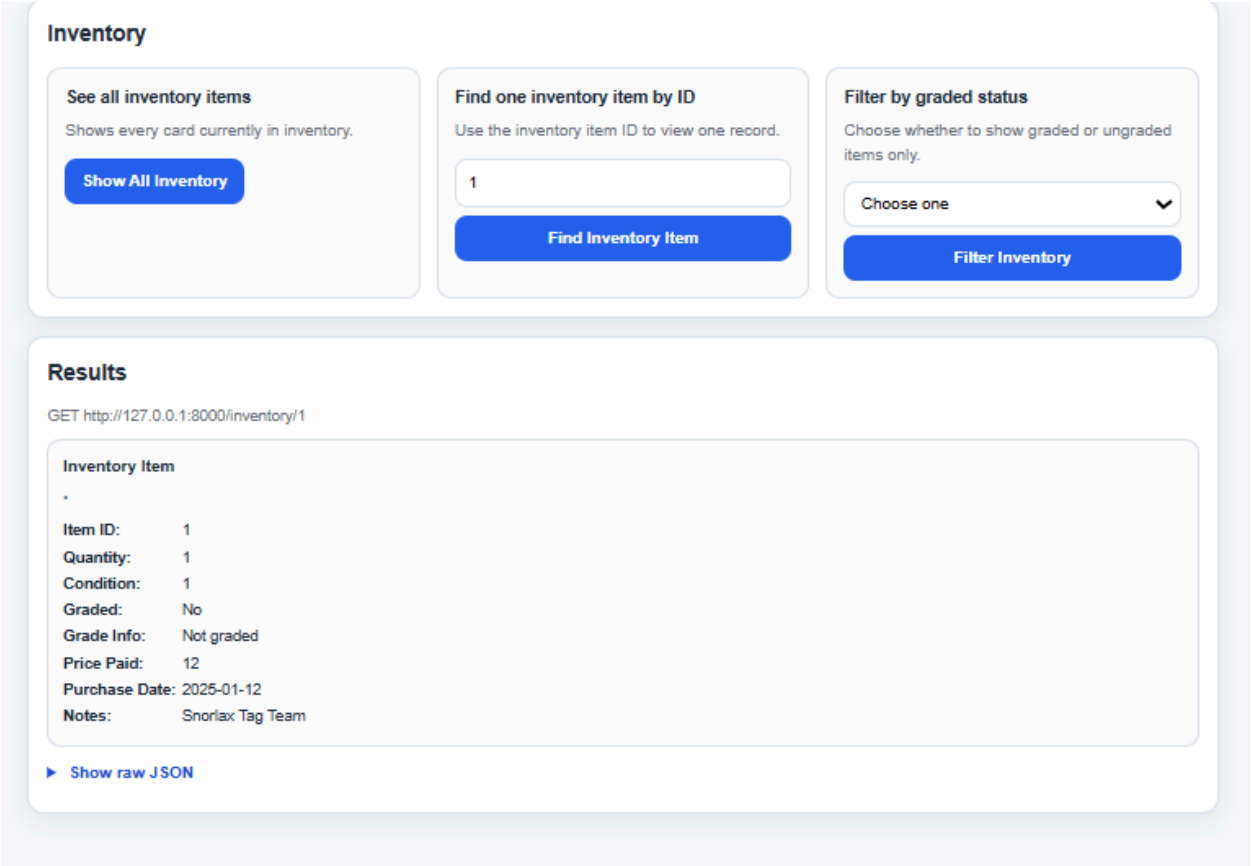
(Screenshot: Swagger showing PUT /inventory/{item_id} request)



(Screenshot: SQLite query showing updated inventory_item record)

```
1|1|1|0|0|||1|12.0|2025-01-12|Snorlax Tag Team
```

(Screenshot: Client displaying updated inventory item)



Test 5 – Delete Inventory Item

Purpose: Verify that inventory records can be removed.

Steps:

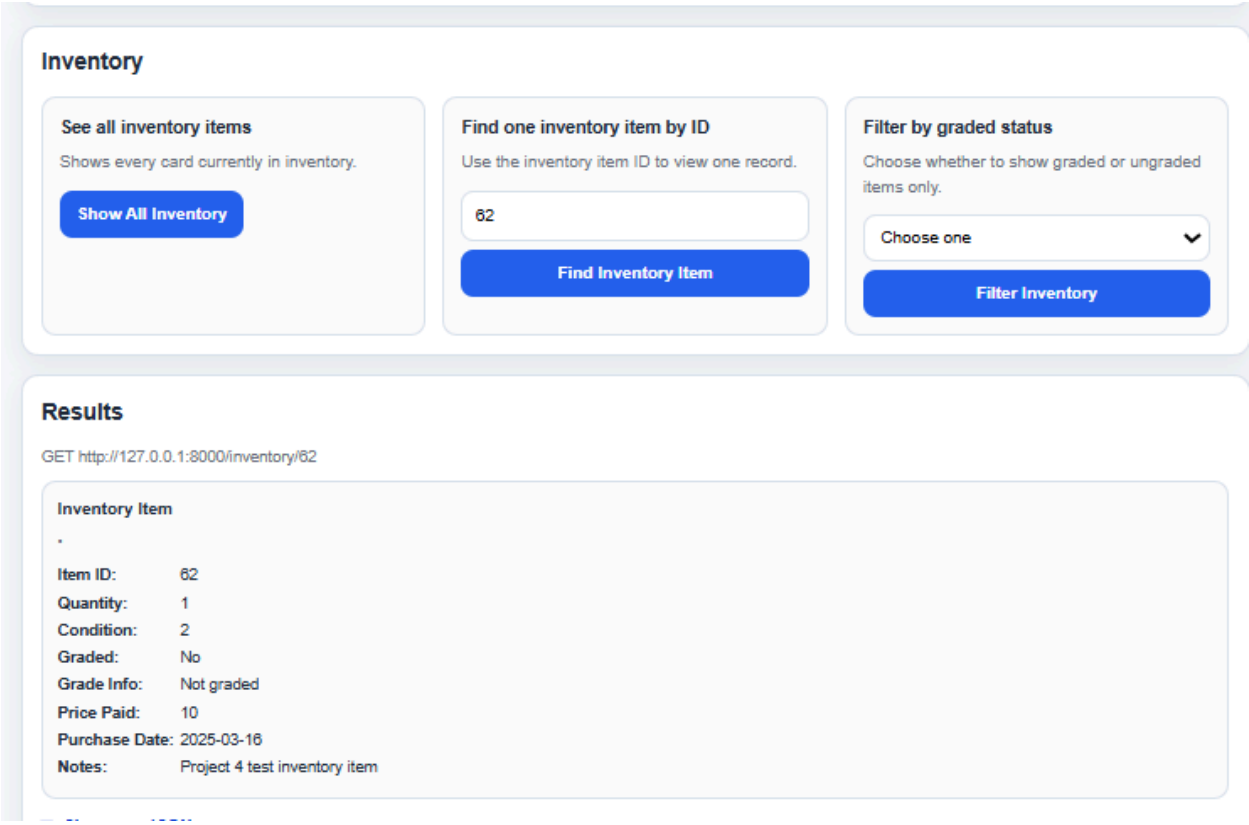
1. Select an inventory item in the client interface.
2. Delete the item.
3. Execute the DELETE /inventory/{item_id} API request.
4. Query the database to confirm the record is removed.

Expected Result:

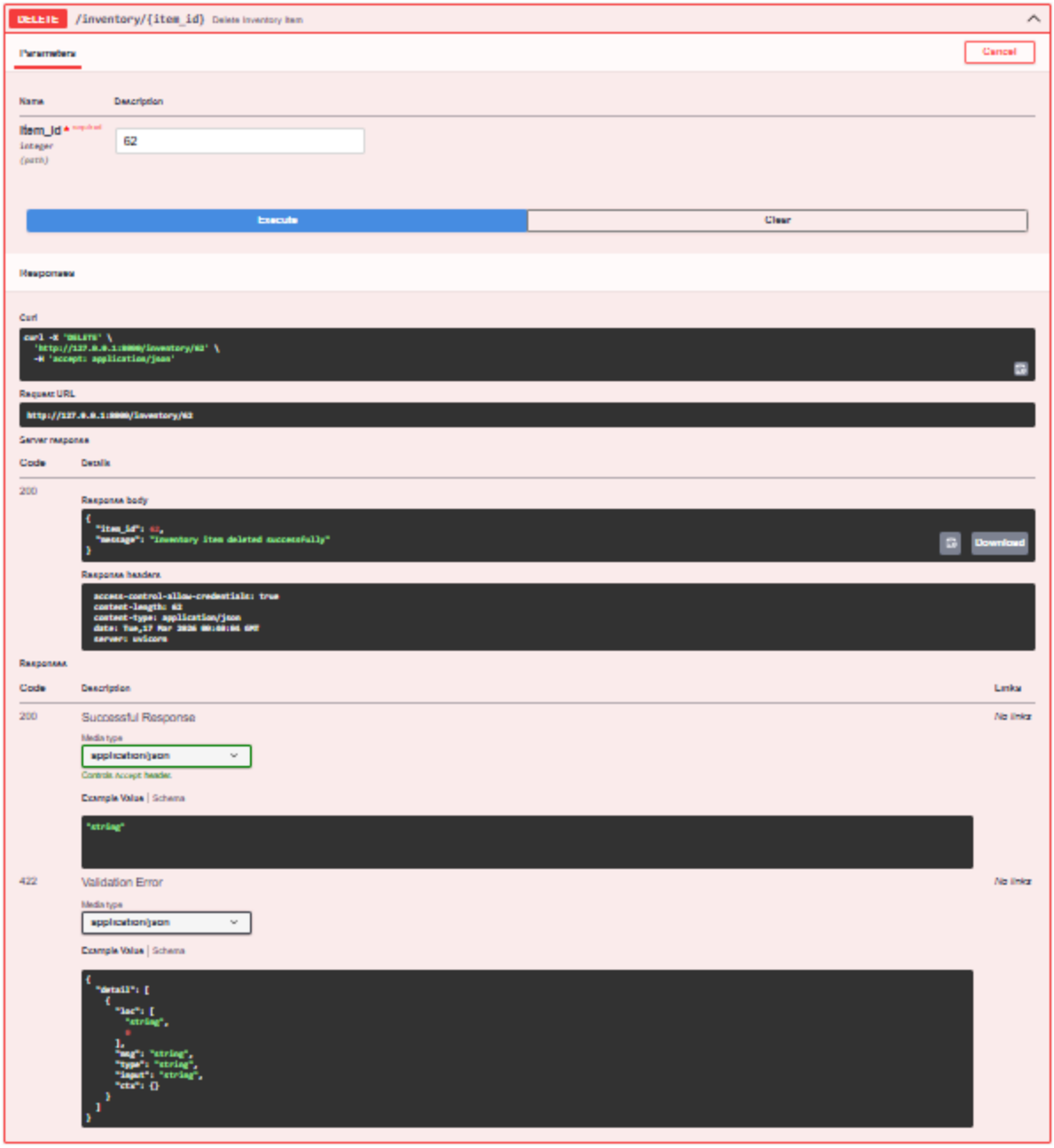
The record disappears from the client interface and database.

Verification:

(Screenshot: Client displaying inventory item before deletion)



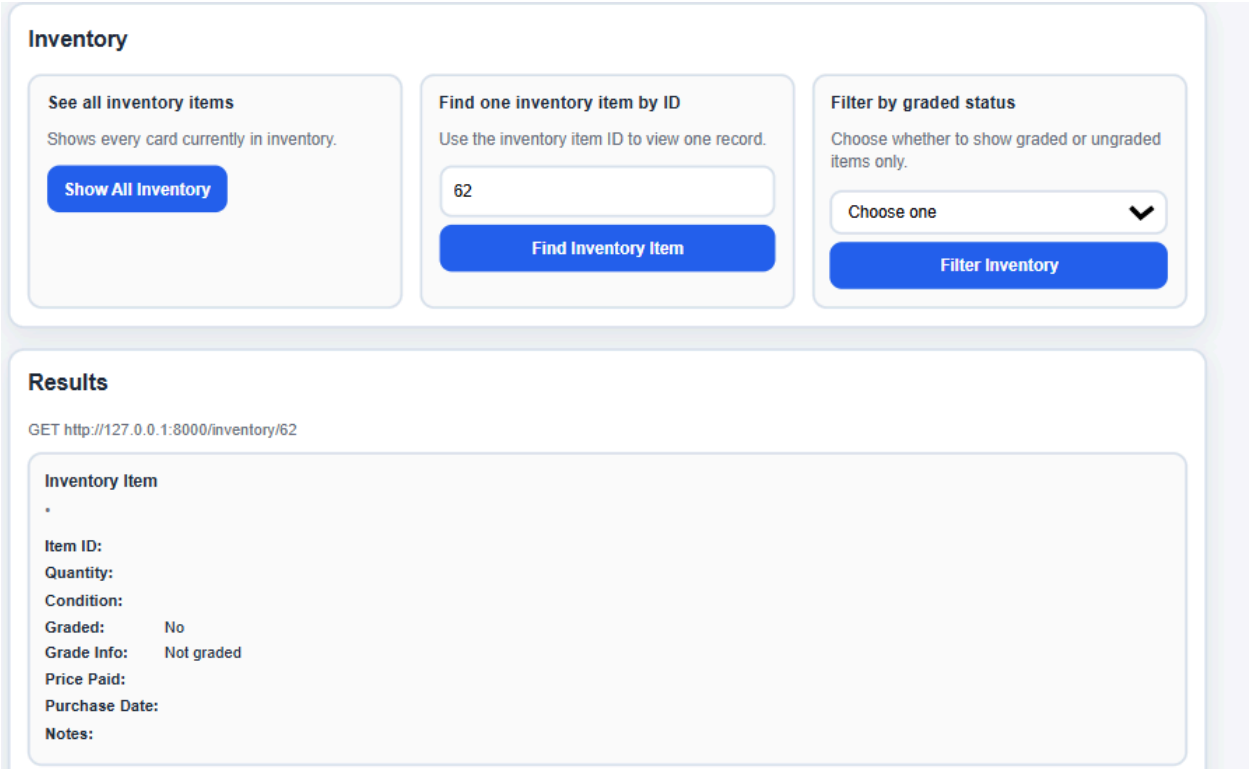
(Screenshot: Swagger showing DELETE /inventory/{item_id} response)



(Screenshot: SQLite query showing no matching record)

```
sqlite> SELECT * FROM inventory_item WHERE item_id = 62;
sqlite>
```

(Screenshot: Client showing inventory list after deletion)



Test Results

All required functionality of the Pokémon Card Tracker application was successfully tested.

The system correctly performs the following operations:

- Retrieve all records
- Retrieve records by ID
- Insert new records
- Update existing records
- Delete records (optional)

Each operation was verified through:

- the client interface
- the REST API service layer
- the SQLite database

These results confirm that the client, service, business, and data layers are correctly integrated and functioning as expected.

Conclusion

The Pokémon Card Tracker application was successfully deployed and tested. The system demonstrates a complete multi-tier architecture with proper communication between the client layer, service layer, business logic layer, and data layer.